

uOttawa

Lab 3 - Traffic Light Controller

CEG3155-Digital Systems II

Fall 2026

School of Electrical Engineering and Computer Science
University of Ottawa

Course Coordinator:
Prof. Rami Abielmona

Teaching Assistants:
Mathis Hurens Barrette

Group: 11
Student Name and number: Marcel Traore 300379484
Student Name and number: Kai Rasco #300304789
Experiment Date:
Submission Date:

Table of contents

Introduction ..... -2-
Problem Discussion ..... -2-
FSM Design ..... -3-
  Problem Specification ..... -3-
  State Diagram ..... -3-
  State Table ..... -3-
  State Minimization ..... -3-
  State Assignment ..... -4-
  Transition Table ..... -4-
  Design Equations ..... -4-
  Circuit Implementation ..... -6-
Discussion of Used Components ..... -7-
Discussion of Actual Solution ..... -8-
Discussion of Tool ..... -10-
Discussion of Challenging Problems ..... -10-
Simulation Results ..... -10-
  Traffic Light Controller Simulation ..... -10-
  Counter Simulation ..... -11-
  Comparator Simulation ..... -12-
  Adder Simulation ..... -12-
  Hex to BCD Decoder Simulation ..... -12-
  Seven-Segment Display Simulation ..... -13-
Verification ..... -13-
Discussion ..... -14-
Appendix: VHDL Code ..... -14-

Introduction

Problem Discussion

Port	Name	Description	On the board
Input	`GClock`	Global clock	1 Hz from `clk_div`
Input	`GReset`	Global reset, asynchronous, active low	`CPU_RESETN`
Input	`MSC[3..0]`	Main street green time	`SW[7..4]`
Input	`SSC[3..0]`	Side street green time	`SW[3..0]`
Input	`SSCS`	Side street car sensor	`BTNC` (debounced)
Output	`MSTL[2..0]`	Main street light, one-hot (100 G, 010 Y, 001 R)	`LED[5..3]`
Output	`SSTL[2..0]`	Side street light, one-hot	`LED[2..0]`
Output	`BCD1[3..0]`	Tens digit of the current count (bonus)	Left 7-segment digit
Output	`BCD2[3..0]`	Ones digit of the current count (bonus)	Right 7-segment digit

Table 1: Input/output specification

FSM Design

Problem Specification

State Diagram

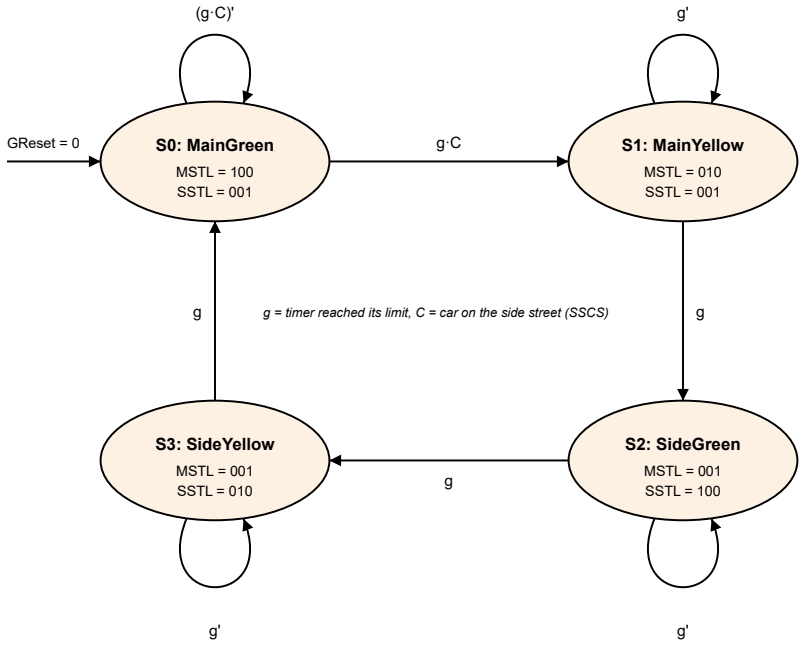


Figure 1: State diagram

State Table

Present state	Next state (gC)				Output		
	00	**01**	**11**	**10**	**MSTL**	**SSTL**	**Timer running**
S0	S0	S0	S1	S0	100	001	MG
S1	S1	S1	S2	S2	010	001	MY
S2	S2	S2	S3	S3	001	100	SG
S3	S3	S3	S0	S0	001	010	SY

Table 2: State table. Only the timer of the current state runs (its reset is the state bit AND `GReset`), the other three are held at 0

State Minimization

With equivalence-class partitioning, Π_1 groups the states that have the same outputs (MSTL, SSTL). Every state has a different pair of lights, so $\Pi_1 = (S_0)(S_1)(S_2)(S_3)$ and $\Pi_2 = \Pi_1$.

State Assignment

State	Sequential (y_1y_0)	Gray (y_1y_0)	One-hot ($y_3y_2y_1y_0$), chosen
S0, Main Green	00	00	0001
S1, Main Yellow	01	01	0010
S2, Side Green	10	11	0100
S3, Side Yellow	11	10	1000

Table 3: State assignment

We chose one hot encoding because each state has its own flip flop, so the lights come directly from the state bits without any decoding (for example $\text{MSTL}(2) = y_0$). The next state equations are also simpler, since each state only depends on the state before it, g and C. The state bits are also used directly to reset the timers of each state. The downside is that it needs 4 flip flops instead of 2, but an FPGA has a lot of flip flops so this is not a problem.

Transition Table

State	Name	$y_3y_2y_1y_0$
S0	Main Green	0001
S1	Main Yellow	0010
S2	Side Green	0100
S3	Side Yellow	1000

Table 4: State codes (one-hot)

Present state	Next state $Y_3Y_2Y_1Y_0$ (gC)				Output	
$y_3y_2y_1y_0$	00	01	11	10	MSTL	SSTL
0001	0001	0001	0010	0001	100	001
0010	0010	0010	0100	0100	010	001
0100	0100	0100	1000	1000	001	100
1000	1000	1000	0001	0001	001	010

Table 5: Transition table. The other 12 codes are unreachable (don't cares)

Design Equations

gC / y_3y_0	00	01	11	10
00	0	1	d	0
01	0	1	d	0
11	0	0	d	1
10	0	1	d	1

Table 6: Y_0: groups $y_0\overline{g}$, $y_0\overline{C}$ and y_3g

gC / y_0y_1	00	01	11	10
00	0	1	d	0
01	0	1	d	0
11	0	0	d	1
10	0	0	d	0

Table 7: Y_1: groups $y_1\overline{g}$ and y_0gC

g / y_1y_2	**00**	**01**	**11**	**10**
0	0	1	d	0
1	0	0	d	1

Table 8: Y_2 (C does not matter)

g / y_2y_3	**00**	**01**	**11**	**10**
0	0	1	d	0
1	0	0	d	1

Table 9: Y_3 (C does not matter)

Next state:

$$Y_0 = y_3 g + y_0\left(\overline{g} + \overline{C}\right) \qquad Y_1 = y_0 g C + y_1 \overline{g}$$

$$Y_2 = y_1 g + y_2 \overline{g} \qquad Y_3 = y_2 g + y_3 \overline{g}$$

Flip-flop 0 stores the inverse: $D_0 = \overline{Y_0}$ and $y_0 = \overline{q_0}$.

Lights:

$$\mathrm{MSTL}(2) = y_0, \quad \mathrm{MSTL}(1) = y_1, \quad \mathrm{MSTL}(0) = y_2 + y_3$$

$$\mathrm{SSTL}(2) = y_2, \quad \mathrm{SSTL}(1) = y_3, \quad \mathrm{SSTL}(0) = y_0 + y_1$$

Counter resets (active low), enable and timer flag:

$$\mathrm{ResetMG} = y_0 \cdot \mathrm{GReset}, \quad \mathrm{ResetMY} = y_1 \cdot \mathrm{GReset}, \quad \mathrm{ResetSG} = y_2 \cdot \mathrm{GReset}, \quad \mathrm{ResetSY} = y_3 \cdot \mathrm{GReset}$$

$$\mathrm{EnableMG} = \overline{\mathrm{gteq_MG}}, \quad g = \mathrm{gteq_MG} + \mathrm{gteq_MY} + \mathrm{gteq_SG} + \mathrm{gteq_SY}$$

BCD decoder (bonus): $s_3 s_2 s_1 s_0$ is the 4-bit count, BCD1 is the tens digit and BCD2 is the ones digit.

Count	$s_3 s_2 s_1 s_0$	**BCD1**	**BCD2**
0	0000	0000	0000
1	0001	0000	0001
2	0010	0000	0010
3	0011	0000	0011
4	0100	0000	0100
5	0101	0000	0101
6	0110	0000	0110
7	0111	0000	0111
8	1000	0000	1000
9	1001	0000	1001
10	1010	0001	0000
11	1011	0001	0001
12	1100	0001	0010
13	1101	0001	0011
14	1110	0001	0100
15	1111	0001	0101

Table 10: Hex to BCD truth table

The counter needs 4 bits to represent all 16 possible numbers (0 to 15), so there are no don't cares. BCD1(3..1) is always 000, so only BCD1(0) needs a K-map. BCD2(0) $= s_0$ because taking away 10 does not change if the number is odd or even, so it does not need one either.

s_3s_2 / s_1s_0	**00**	**01**	**11**	**10**
00	0	0	0	0
01	0	0	0	0
11	1	1	1	1
10	0	0	1	1

Table 11: BCD1(0): groups s_3s_2 and s_3s_1

s_3s_2 / s_1s_0	**00**	**01**	**11**	**10**
00	0	0	0	0
01	0	0	0	0
11	0	0	0	0
10	1	1	0	0

Table 12: BCD2(3): group $s_3\overline{s_2s_1}$

s_3s_2 / s_1s_0	**00**	**01**	**11**	**10**
00	0	0	0	0
01	1	1	1	1
11	0	0	1	1
10	0	0	0	0

Table 13: BCD2(2): groups $\overline{s_3}s_2$ and s_2s_1

s_3s_2 / s_1s_0	**00**	**01**	**11**	**10**
00	0	0	1	1
01	0	0	1	1
11	1	1	0	0
10	0	0	0	0

Table 14: BCD2(1): groups $\overline{s_3}s_1$ and $s_3s_2\overline{s_1}$

From the K-maps, we get

$$\text{BCD1}(0) = s_3s_2 + s_3s_1$$

$$\text{BCD2}(3) = s_3\overline{s_2s_1}, \quad \text{BCD2}(2) = \overline{s_3}s_2 + s_2s_1, \quad \text{BCD2}(1) = \overline{s_3}s_1 + s_3s_2\overline{s_1}, \quad \text{BCD2}(0) = s_0$$

Circuit Implementation

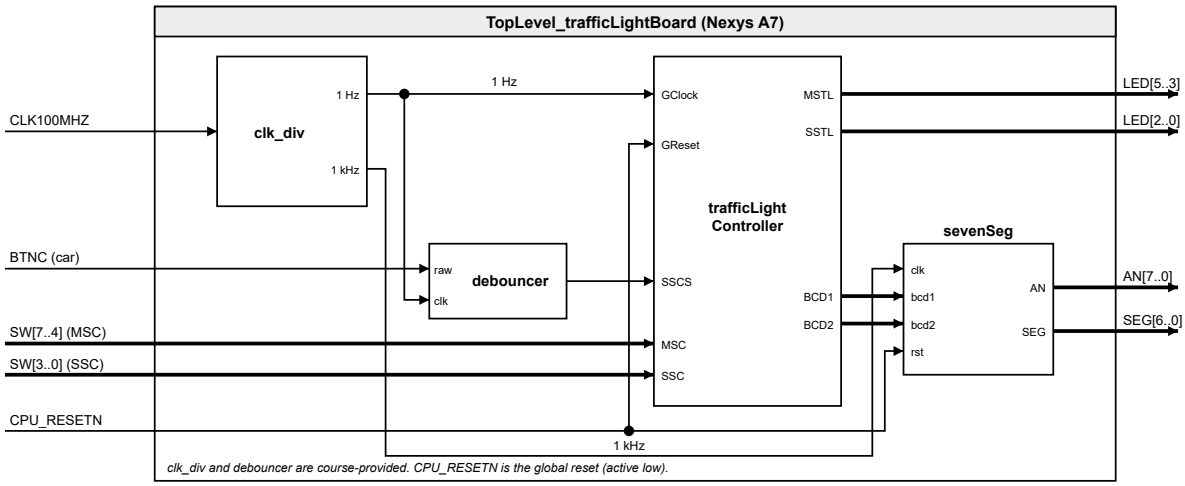


Figure 2: Top level on the Nexys A7

The circuit is done in structural VHDL, using the components given by the course and our own components. The state register is 4 `enARdFF_2` flip flops, one for each state, clocked by `GClock` and reset by `GReset`. The next state equations Y_0 to Y_3 and the light outputs are done with AND, OR and NOT gates, directly from the design equations. Flip flop 0 gets $\overline{Y_0}$ and its output is inverted again to give y_0, so the reset puts the FSM in S0.

The four timers are built the same way. Each `counter4` is 4 `enARdFF_2` flip flops and an `adder4` that adds 1 to the count at every clock. The `adder4` is a chain of 4 `oneBitAdder`. Each `comparator4` is a chain of 4 `oneBitComparator` starting from the most significant bit, and gteq is the inverse of lt. The hex to BCD decoder is made of gates from the BCD equations above.

The top level connects the controller to the board. The 100 MHz clock goes into `clk_div`, which gives 1 Hz for the controller and the debouncer, and 1 kHz for the display. The car button `BTNC` goes through the debouncer before going to `SSCS`. `CPU_RESETN` is the global reset (active low) and goes to the controller and the display. The switches `SW[7..4]` and `SW[3..0]` give `MSC` and `SSC`, and the lights go to `LED[5..3]` for the main street and `LED[2..0]` for the side street. The display driver `sevenSeg` switches between the two digits at 1 kHz to show `BCD1` and `BCD2`. The pins are set in the constraints file `lab3_fsm.xdc`.

Discussion of Used Components

Everything is structural RTL VHDL; only the course flip-flop and clock divider are behavioral. The main code is in the Appendix.

TopLevel_trafficLightBoard	Board top level: clock divider, debouncer, controller and display driver, mapped to the Nexys A7 pins.
trafficLightController	Top-level entity of the lab: 4-state one-hot FSM, four timers, OR gates and BCD decoder.
counter4	4-bit up counter with enable and asynchronous clear: 4 `enARdFF_2` and `adder4` adding 1.
comparator4	4-bit unsigned comparator: a chain of 4 `oneBitComparator` starting from the most significant bit. Gives gt, lt and gteq = not lt.
adder4	4-bit ripple carry adder from 4 `oneBitAdder`.
hex_bcd_decoder	Turns the 4-bit count into tens and ones BCD digits (bonus), from K-map equations.
sevenSeg	Shows the two digits on the Nexys display: one flip-flop switches between the digits at 1 kHz, a multiplexer picks the digit and `dec_7seg` drives the segments.
enARdFF_2, oneBitAdder, oneBitComparator	Provided by the course: D flip-flop with enable and asynchronous reset, 1-bit full adder, 1-bit comparator.
clk_div, debouncer, dec_7seg	Provided by the course: clock divider (changed for the 100 MHz clock), push-button debouncer, 7-segment decoder.

Discussion of Actual Solution

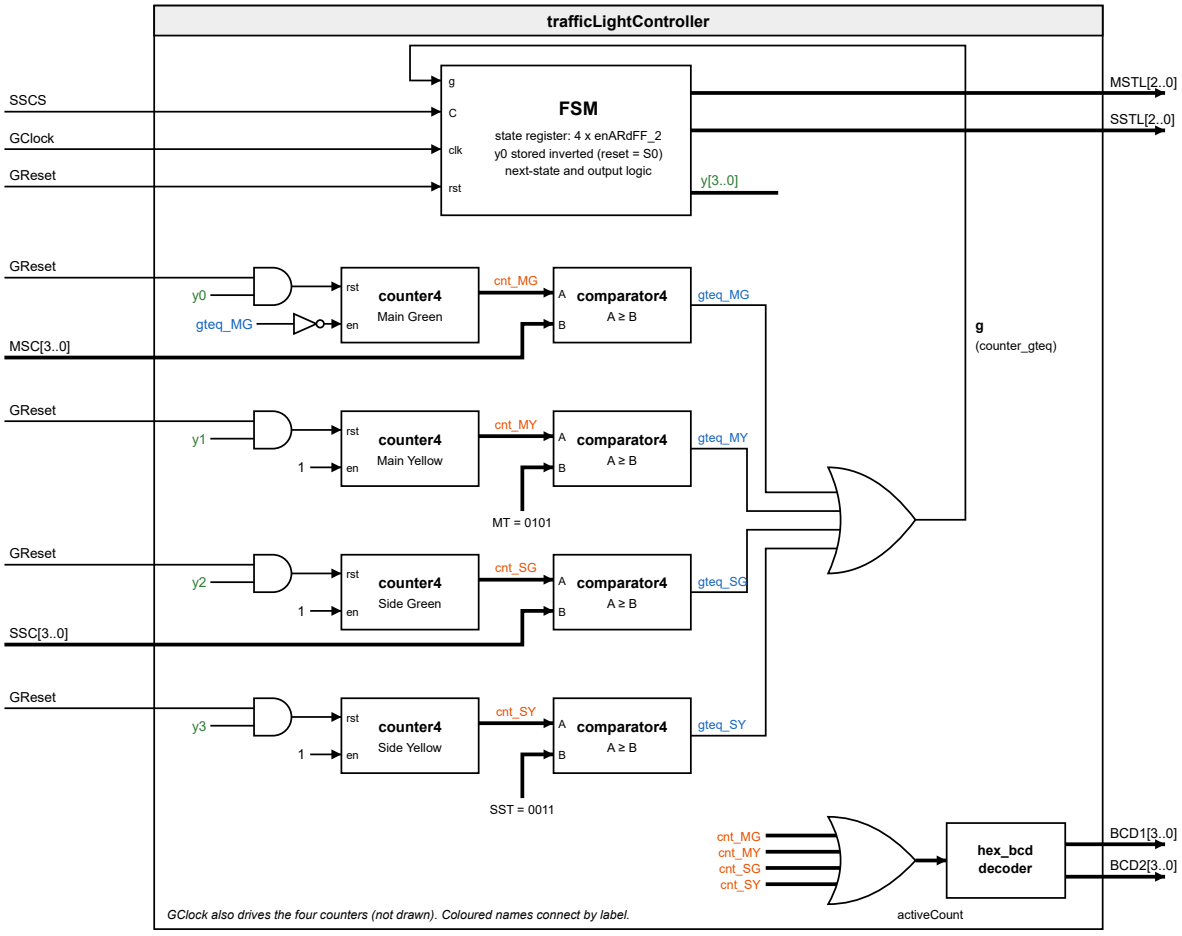


Figure 3: Traffic light controller block diagram

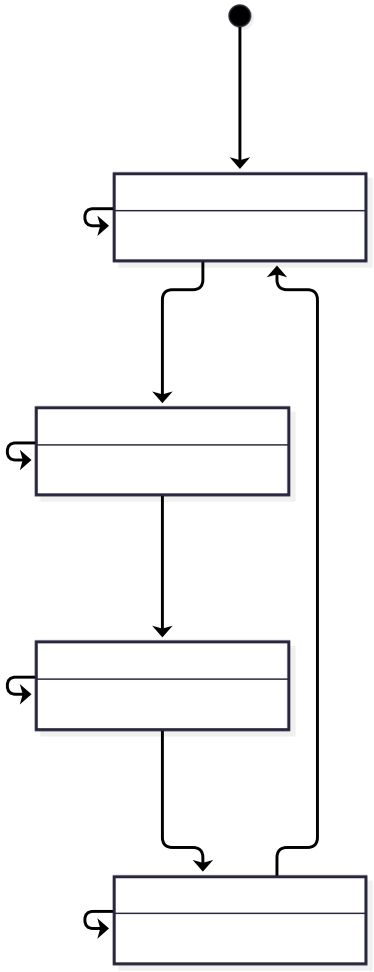


Figure 4: Flowchart of the solution

Our solution is a Moore FSM with 4 states, main green (S0), main yellow (S1), side green (S2) and side yellow (S3). We used one hot encoding, so each state has its own flip flop (y_0 to y_3) and the lights come directly from the state bits. y_0 is stored inverted, so when `GReset` is pressed all the flip flops go to 0 and the FSM starts in S0 with the main street green and the side street red.

Each state has its own timer made of a `counter4` and a `comparator4`. A timer only counts when its state is active, because its reset is the state bit AND `GReset`. The other three timers stay at 0. The comparator checks if the count reached the limit. `MSC` and `SSC` come from the switches and the yellow times are fixed in the code (`MT` = 5 and `SST` = 3). The comparator outputs are combined into g, which tells the FSM that the time of the current state is over.

When g is 1, the FSM goes to the next state. The only exception is the main green, which only goes to yellow if g is 1 and there is a car (C). If there is no car, the main green counter stops at `MSC` because its enable is $\overline{\text{gteq_MG}}$, and the light stays green until a car arrives.

For the bonus, the four counts are OR'ed together into `activeCount`. Since only one counter runs at a time, this is basically the count of the current state. The hex to BCD decoder splits it into the tens (`BCD1`) and the ones (`BCD2`), which are shown on the display.

On the board, the top level adds the clock divider (1 Hz for the controller and 1 kHz for the display), the debouncer for the car button and the `sevenSeg`.

Discussion of Tool

We simulated everything with GHDL and looked at the waveforms (`.ghw`) in GTKWave. A PowerShell script (`run_tests.ps1`) compiles the design and runs all the testbenches. This script was created using AI to make it faster, but could also be done without this script.

Discussion of Challenging Problems

We had two main problems. First, the synthesis in Vivado failed with a system resources error. It was not from our code but from Vivado and Windows, and it worked after restarting the computer and running it again. Second, the anodes of the 7-segment display were confusing to use as a selector for the digit number. All 8 digits share the same segment pins, so the anodes are basically used to choose which digit is on. A flip-flop switches between the two digits at 1 kHz, which is fast enough that both look on at the same time.

Simulation Results

The clock period in the simulations is 10 ns, so 100 ns is 10 cycles. Each phase lasts its limit + 1 cycle.

Traffic Light Controller Simulation

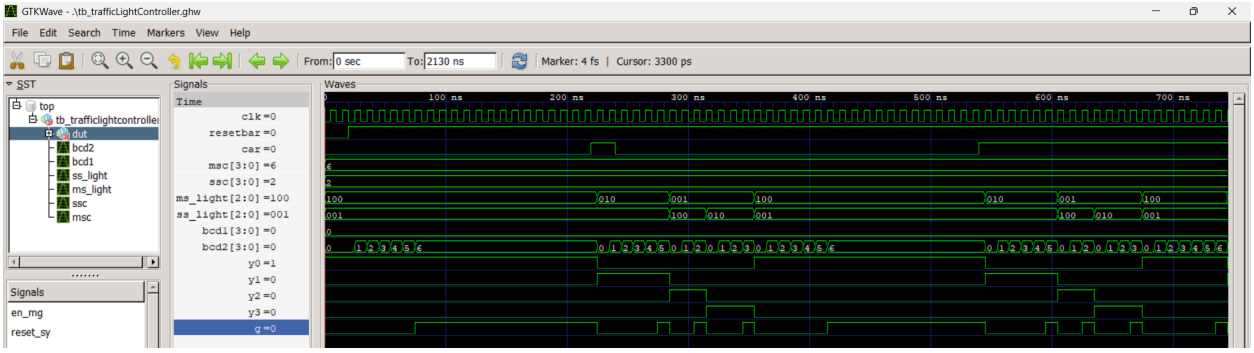


Figure 5: 0 to 740 ns. During reset the lights are already MS green and SS red (S0). With no car, the count goes from 1 to 6 and stays at 6 ($g = 1$). A car at 220 ns gives yellow on the next clock, then MS yellow lasts 6 cycles, SS green 3 (`SSC` = 2) and SS yellow 4 (`SST` = 3).

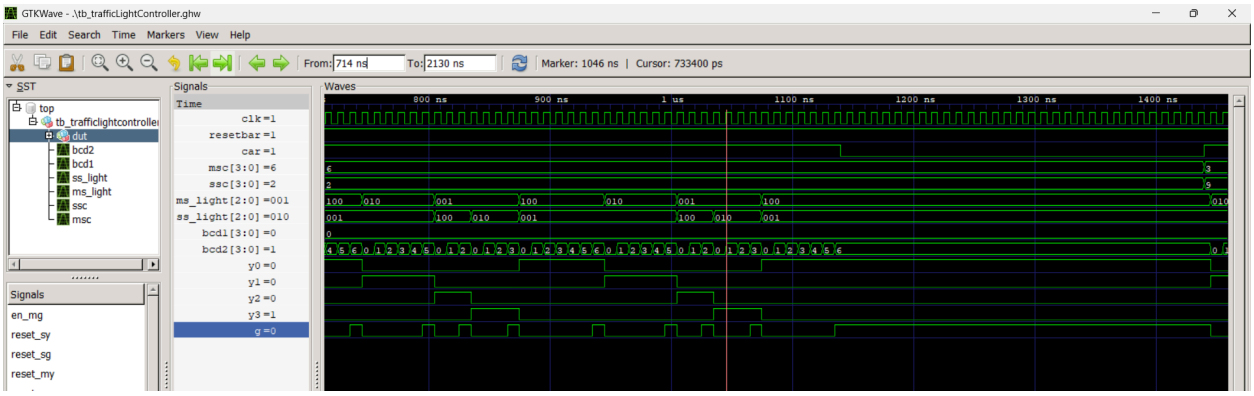


Figure 6: 714 to 1460 ns. The car waits for two full cycles, and MS green lasts 7 cycles (`MSC` + 1) each time. When the car leaves at 1140 ns, MS stays green and the count stays at 6.

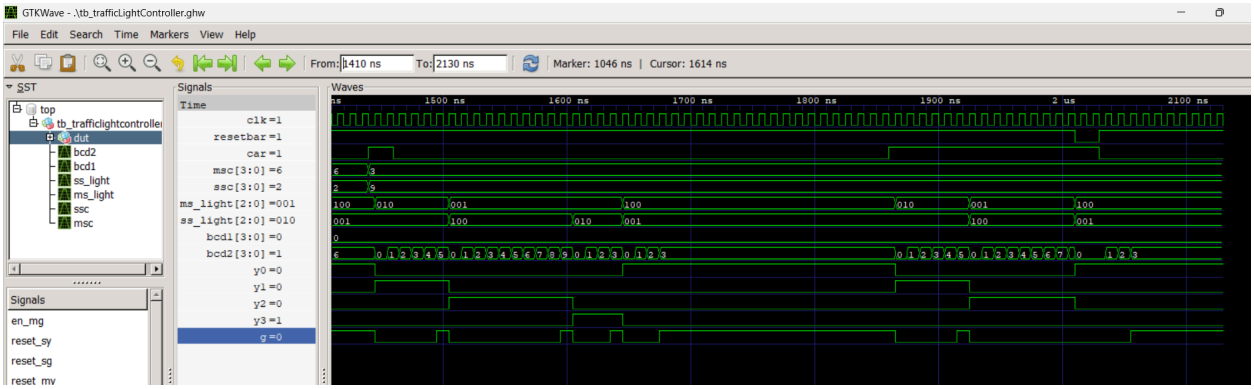


Figure 7: 1410 to 2130 ns. With `MSC` = 3 and `SSC` = 9, SS green lasts 10 cycles and MS green stops counting at 3. The reset at 2010 ns goes back to S0 right away, even in the middle of SS green.

Counter Simulation

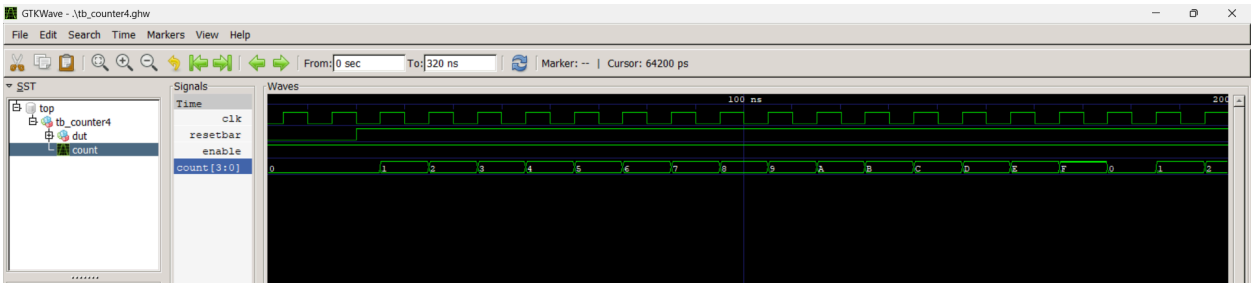


Figure 8: `counter4`: after the reset is released, it counts from 0 to F and wraps back to 0.

Comparator Simulation

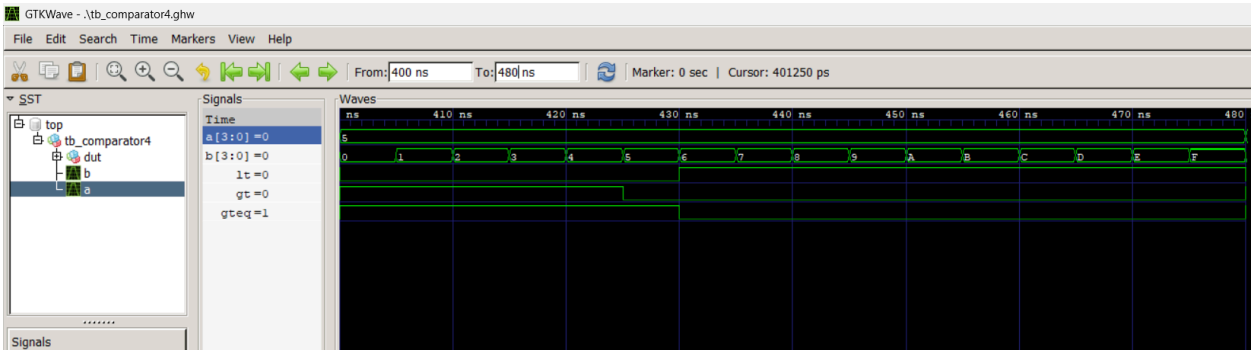


Figure 9: `comparator4` with A = 5: gt while B < 5, both gt and lt are 0 at B = 5, lt from B = 6, and gteq = not lt.

Adder Simulation

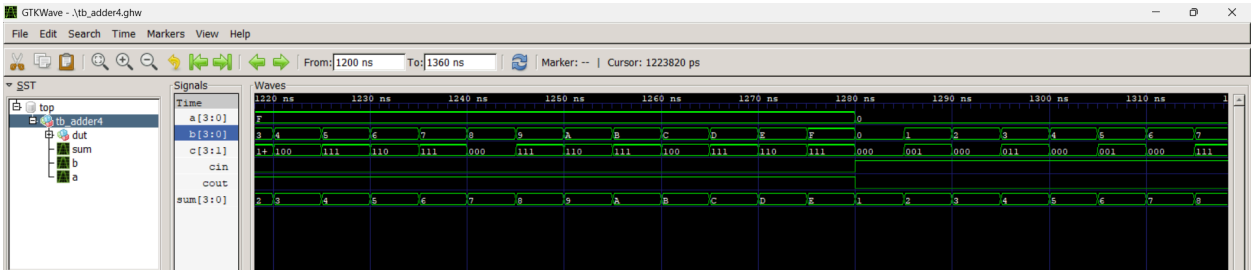


Figure 10: `adder4` from 1200 to 1360 ns: with A = F the sum wraps and cout = 1. From 1280 ns, A = 0 and cin = 1, so the sum is B + 1.

Hex to BCD Decoder Simulation

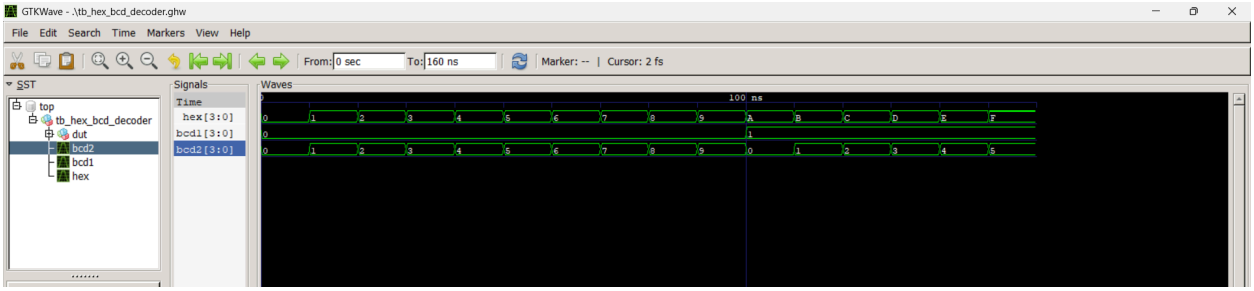


Figure 11: `hex_bcd_decoder` with inputs 0 to F. From A (10), bcd1 (tens) is 1 and bcd2 (ones) starts again at 0.

Seven-Segment Display Simulation

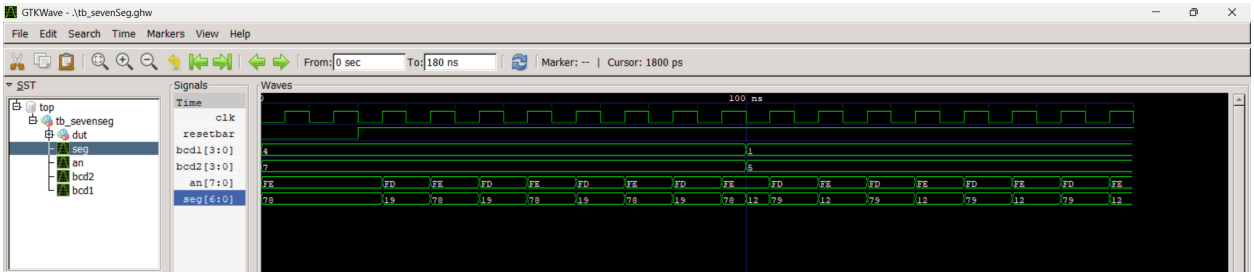


Figure 12: `sevenSeg`: `an` switches between FE (right digit) and FD (left digit) on every clock. The segments show 7 and 4 (47), then 5 and 1 (15). They are active low: 78 is 7, 19 is 4, 12 is 5 and 79 is 1.

Verification

Every component has a testbench that applies the cases below. We compared the waveforms with the expected values from our tables and equations, and all of them matched.

Testbench	Cases	Result
tb_adder4	All 512 combinations of A, B and cin	Pass
tb_comparator4	All 256 pairs of A and B	Pass
tb_counter4	Reset, counting with wrap-around, hold with enable = 0, reset	Pass
tb_hex_bcd_decoder	All 16 inputs	Pass
tb_sevenSeg	Two numbers, digit switching and segments	Pass
tb_trafficLightController	Reset, no car, late car, car waiting, car leaving early, new switch values, reset during SS green	Pass

Table 15: Verification summary

Phase	Expected (cycles)	Simulation
MS green, car waiting, `MSC` = 6	7	7
MS green, car arrives after `MSC`	1 more	1
MS yellow, `MT` = 5	6	6
SS green, `SSC` = 2	3	3
SS green, `SSC` = 9	10	10
SS yellow, `SST` = 3	4	4
Reset during SS green	S0 right away	S0 right away

Table 16: Expected and simulated phase lengths

We were able to present the demonstration by uploading the code via Vivado. We were able to confirm the waveform that we got during the simulations.

Discussion

The design worked in simulation and on the FPGA board in the lab. The limitations are that `MSC` and `SSC` cannot be 0, that each phase lasts one cycle longer than its limit, and that the car has to be present (button held) when the main green time is over.

A possible improvement would be to lower the debounce time, so we do not have to press the button so long.

Appendix: VHDL Code

The main components are listed below. The testbenches and the course-provided files are in the submitted source files.

TopLevel_trafficLightBoard.vhdl

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity TopLevel_trafficLightBoard is
    port (
        CLK100MHZ  : in  std_logic;
        CPU_RESETN : in  std_logic;
-- global reset, active low
        BTNC       : in  std_logic;
-- car sensor
        SW         : in  std_logic_vector(7 downto 0);
-- 7-4 = MSC, 3-0 = SSC

        LED        : out std_logic_vector(5 downto 0);
-- 5-3 = MSTL, 2-0 = SSTL
        AN         : out std_logic_vector(7 downto 0);
        SEG        : out std_logic_vector(6 downto 0)
    );
end entity TopLevel_trafficLightBoard;

architecture structural of TopLevel_trafficLightBoard
is

    component clk_div is
        port (
            clock_100Mhz : in  std_logic;
            clock_1MHz, clock_100KHz, clock_10KHz,
clock_1KHz,
            clock_100Hz, clock_10Hz, clock_1Hz : out
std_logic
        );
    end component;

    component debouncer is
        port (
            i_raw   : in  std_logic;
            i_clock : in  std_logic;
            o_clean : out std_logic
        );
    end component;

    component trafficLightController is
        port (
            GClock : in  std_logic;
            GReset : in  std_logic;
            SSCS   : in  std_logic;
            MSC    : in  std_logic_vector(3 downto 0);
            SSC    : in  std_logic_vector(3 downto 0);
            MSTL   : out std_logic_vector(2 downto 0);
            SSTL   : out std_logic_vector(2 downto 0);
            BCD1   : out std_logic_vector(3 downto 0);
            BCD2   : out std_logic_vector(3 downto 0)
        );
    end component;

    component sevenSeg is
        port (
            i_clk      : in  std_logic;
            i_resetBar : in  std_logic;
            i_bcd1     : in  std_logic_vector(3 downto
0);
            i_bcd2     : in  std_logic_vector(3 downto
0);
            o_an       : out std_logic_vector(7 downto
0);
            o_seg      : out std_logic_vector(6 downto
0)
        );
    end component;

    signal s_1Hz  : std_logic;   -- controller and
debouncer clock
    signal s_1kHz : std_logic;   -- display clock
    signal s_car  : std_logic;   -- debounced car
sensor
    signal s_bcd1 : std_logic_vector(3 downto 0);
    signal s_bcd2 : std_logic_vector(3 downto 0);

begin

    -- 1) Clock divider: 100 MHz -> 1 kHz (display) and
1 Hz (controller)
    CLKDIV_INST: clk_div
        port map (
            clock_100Mhz => CLK100MHZ,
            clock_1MHz   => open,
            clock_100KHz => open,
            clock_10KHz  => open,
            clock_1KHz   => s_1kHz,
            clock_100Hz  => open,
            clock_10Hz   => open,
            clock_1Hz    => s_1Hz
```

```
        );

    -- 2) Debounce the car sensor, on the same clock as
the controller
    DEB_INST: debouncer
        port map (
            i_raw   => BTNC,
            i_clock => s_1Hz,
            o_clean => s_car
        );

    -- 3) Traffic light controller
    FSM_INST: trafficLightController
        port map (
            GClock => s_1Hz,
            GReset => CPU_RESETN,
            SSCS   => s_car,
            MSC    => SW(7 downto 4),
            SSC    => SW(3 downto 0),
            MSTL   => LED(5 downto 3),
            SSTL   => LED(2 downto 0),
            BCD1   => s_bcd1,
            BCD2   => s_bcd2
        );

    -- 4) 7-segment display: tens on the left digit,
ones on the right
    DISP_INST: sevenSeg
        port map (
            i_clk      => s_1kHz,
            i_resetBar => CPU_RESETN,
            i_bcd1     => s_bcd1,
            i_bcd2     => s_bcd2,
            o_an       => AN,
            o_seg      => SEG
        );

end architecture structural;
```

trafficLightController.vhdl

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity trafficLightController is
    port (
        GClock : in  std_logic;
        GReset : in  std_logic;
        SSCS   : in  std_logic;
        MSC    : in  std_logic_vector(3 downto 0);
        SSC    : in  std_logic_vector(3 downto 0);

        MSTL   : out std_logic_vector(2 downto 0);
        SSTL   : out std_logic_vector(2 downto 0);

        BCD1   : out std_logic_vector(3 downto 0);
        BCD2   : out std_logic_vector(3 downto 0)
    );
end entity trafficLightController;

architecture structural of trafficLightController is

    -- Yellow light set time (5 seconds and 3 seconds)
    constant MT  : std_logic_vector(3 downto 0) :=
"0101";
    constant SST : std_logic_vector(3 downto 0) :=
"0011";

    component enardFF_2 is
        port (
            i_resetBar : in  std_logic;
            i_d        : in  std_logic;
            i_enable   : in  std_logic;
            i_clock    : in  std_logic;
            o_q        : out std_logic;
            o_qbar     : out std_logic
        );
    end component;

    component counter4 is
        port (
            i_clock    : in  std_logic;
            i_resetBar : in  std_logic;
            i_enable   : in  std_logic;
            o_count    : out std_logic_vector(3 downto
0)
        );
    end component;

    component comparator4 is
        port (
            i_a    : in  std_logic_vector(3 downto 0);
            i_b    : in  std_logic_vector(3 downto 0);
            o_gt   : out std_logic;
            o_lt   : out std_logic;
            o_gteq : out std_logic
        );
```

```
    end component;

    component hex_bcd_decoder is
        port (
            i_hex  : in  std_logic_vector(3 downto 0);
            o_bcd1 : out std_logic_vector(3 downto 0);
            o_bcd2 : out std_logic_vector(3 downto 0)
        );
    end component;

    -- State bits
    signal y3, y2, y1, y0 : std_logic;
    signal q0             : std_logic;

    -- Next-state signals
    signal Y3_in, Y2_in, Y1_in, Y0_in : std_logic;
    signal D_FF0                      : std_logic;

    -- Counter values
    signal count_MG, count_MY, count_SG, count_SY :
std_logic_vector(3 downto 0);

    -- Comparator flags
    signal gteq_MG, gteq_MY, gteq_SG, gteq_SY :
std_logic;
    signal g                                  :
std_logic;

    -- Counter controls
    signal reset_MG, reset_MY, reset_SG, reset_SY :
std_logic;
    signal en_MG                                  :
std_logic;

    -- Internal active counter value
    signal activeCount : std_logic_vector(3 downto 0);

begin


    y0    <= not q0;
    D_FF0 <= not Y0_in;

    FF0: enardFF_2
        port map (
            i_resetBar => GReset,
            i_d        => D_FF0,
            i_enable   => '1',
            i_clock    => GClock,
            o_q        => q0,
            o_qbar     => open
        );

    FF1: enardFF_2
        port map (
            i_resetBar => GReset,
            i_d        => Y1_in,
            i_enable   => '1',
            i_clock    => GClock,
            o_q        => y1,
            o_qbar     => open
        );

    FF2: enardFF_2
        port map (
            i_resetBar => GReset,
            i_d        => Y2_in,
            i_enable   => '1',
            i_clock    => GClock,
            o_q        => y2,
            o_qbar     => open
        );

    FF3: enardFF_2
        port map (
            i_resetBar => GReset,
            i_d        => Y3_in,
            i_enable   => '1',
            i_clock    => GClock,
            o_q        => y3,
            o_qbar     => open
        );


    Y0_in <= (y3 and g) or (y0 and ((not g) or (not
SSCS)));
    Y1_in <= (y0 and g and SSCS) or (y1 and (not g));
    Y2_in <= (y1 and g) or (y2 and (not g));
    Y3_in <= (y2 and g) or (y3 and (not g));


    MSTL(2) <= y0;             -- Green
    MSTL(1) <= y1;             -- Yellow
    MSTL(0) <= y2 or y3;       -- Red

    SSTL(2) <= y2;             -- Green
    SSTL(1) <= y3;             -- Yellow
    SSTL(0) <= y0 or y1;       -- Red


    reset_MG <= y0 and GReset;
    reset_MY <= y1 and GReset;
    reset_SG <= y2 and GReset;
    reset_SY <= y3 and GReset;

    en_MG <= not gteq_MG;


    C_MG: counter4
        port map (
            i_clock    => GClock,
            i_resetBar => reset_MG,
            i_enable   => en_MG,
            o_count    => count_MG
        );

    C_MY: counter4
        port map (
            i_clock    => GClock,
            i_resetBar => reset_MY,
            i_enable   => '1',
            o_count    => count_MY
        );

    C_SG: counter4
        port map (
            i_clock    => GClock,
            i_resetBar => reset_SG,
            i_enable   => '1',
            o_count    => count_SG
        );

    C_SY: counter4
        port map (
            i_clock    => GClock,
            i_resetBar => reset_SY,
            i_enable   => '1',
            o_count    => count_SY
        );
```

```
    CMP_MG: comparator4
        port map (
            i_a    => count_MG,
            i_b    => MSC,
            o_gt   => open,
            o_lt   => open,
            o_gteq => gteq_MG
        );

    CMP_MY: comparator4
        port map (
            i_a    => count_MY,
            i_b    => MT,
            o_gt   => open,
            o_lt   => open,
            o_gteq => gteq_MY
        );

    CMP_SG: comparator4
        port map (
            i_a    => count_SG,
            i_b    => SSC,
            o_gt   => open,
            o_lt   => open,
            o_gteq => gteq_SG
        );

    CMP_SY: comparator4
        port map (
            i_a    => count_SY,
            i_b    => SST,
            o_gt   => open,
            o_lt   => open,
            o_gteq => gteq_SY
        );

    -- only the counter of the active state can raise g
    -- (an idle counter sits at 0, and 0 >= 0 would be
true when MSC or SSC = 0)
    g <= (y0 and gteq_MG) or (y1 and gteq_MY) or (y2
and gteq_SG) or (y3 and gteq_SY);


    activeCount <= count_MG or count_MY or count_SG or
count_SY;

    BCD_DEC: hex_bcd_decoder
        port map (
            i_hex  => activeCount,
            o_bcd1 => BCD1,
            o_bcd2 => BCD2
        );



end architecture structural;
```

counter4.vhdl

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity counter4 is
    port (
        i_clock    : in  std_logic;
        i_resetBar : in  std_logic;
        i_enable   : in  std_logic;
        o_count    : out std_logic_vector(3 downto 0)
    );
end entity counter4;

architecture structural of counter4 is

    component enardFF_2 is
        PORT(
            i_resetBar  : IN  STD_LOGIC;
            i_d   : IN  STD_LOGIC;
```

```
            i_enable  : IN  STD_LOGIC;
            i_clock   : IN  STD_LOGIC;
            o_q, o_qBar : OUT STD_LOGIC);
    end component;

    component adder4 is
        port (
            i_a    : in  std_logic_vector(3 downto 0);
            i_b    : in  std_logic_vector(3 downto 0);
            i_cin  : in  std_logic;
            o_sum  : out std_logic_vector(3 downto 0);
            o_cout : out std_logic
        );
    end component;

    signal s_count    : std_logic_vector(3 downto 0);
    signal s_next_sum : std_logic_vector(3 downto 0);

begin
    ADDER_INST: adder4
        port map ( i_a => s_count, i_b => "0000", i_cin
=> '1',
                   o_sum => s_next_sum, o_cout =>
open );

    GEN_FF: for i in 0 to 3 generate
        FF_i: enardFF_2
            port map (
                i_resetBar => i_resetBar,
                i_d        => s_next_sum(i),
                i_enable   => i_enable,
                i_clock    => i_clock,
                o_q        => s_count(i),
                o_qbar     => open
            );
    end generate GEN_FF;

    o_count <= s_count;
end architecture structural;
```

comparator4.vhdl

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity comparator4 is
    port (
        i_a, i_b : in  std_logic_vector(3 downto 0);
        o_gt     : out std_logic;
        o_lt     : out std_logic;
        o_gteq   : out std_logic
    );
end entity comparator4;

architecture structural of comparator4 is

    component onebitcomparator is
        PORT(
            i_GTPrevious, i_LTPrevious  : IN
STD_LOGIC;
            i_Ai, i_Bi      : IN  STD_LOGIC;
            o_GT, o_LT      : OUT STD_LOGIC
        );
    end component;

    signal gt, lt: std_logic_vector(4 downto 0);
begin
gt(4) <= '0';
lt(4) <= '0';
```

```
COMP3: oneBitComparator port map (i_GTPrevious =>
gt(4), i_LTPrevious => lt(4),
                                  i_Ai => i_a(3), i_Bi
=> i_b(3),
                                  o_GT => gt(3), o_LT
=> lt(3));
COMP2: oneBitComparator port map (i_GTPrevious =>
gt(3), i_LTPrevious => lt(3),
                                  i_Ai => i_a(2), i_Bi
=> i_b(2),
                                  o_GT => gt(2), o_LT
=> lt(2));

COMP1: oneBitComparator port map (i_GTPrevious =>
gt(2), i_LTPrevious => lt(2),
                                  i_Ai => i_a(1), i_Bi
=> i_b(1),
                                  o_GT => gt(1), o_LT
=> lt(1));
COMP0: oneBitComparator port map (i_GTPrevious =>
gt(1), i_LTPrevious => lt(1),
                                  i_Ai => i_a(0), i_Bi
=> i_b(0),
                                  o_GT => gt(0), o_LT
=> lt(0));

    o_gt   <= gt(0);
```

```
    o_lt   <= lt(0);
    o_gteq <= not lt(0);
end architecture structural;
```

adder4.vhdl

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity adder4 is
    port (
        i_a    : in  std_logic_vector(3 downto 0);
        i_b    : in  std_logic_vector(3 downto 0);
        i_cin  : in  std_logic;
        o_sum  : out std_logic_vector(3 downto 0);
        o_cout : out std_logic
    );
end entity adder4;

architecture structural of adder4 is

    component onebitadder is
         port (
             i_CarryIn         : in  std_logic;
             i_Ai, i_Bi        : in  std_logic;
             o_Sum, o_CarryOut : out std_logic
         );
    end component;

    signal c : std_logic_vector(3 downto 1);

begin
    FA0: onebitadder port map ( i_Ai=>i_a(0),
i_Bi=>i_b(0), i_CarryIn=>i_cin, o_Sum=>o_sum(0),
o_CarryOut=>c(1) );
    FA1: onebitadder port map ( i_Ai=>i_a(1),
i_Bi=>i_b(1), i_CarryIn=>c(1),   o_Sum=>o_sum(1),
o_CarryOut=>c(2) );
    FA2: onebitadder port map ( i_Ai=>i_a(2),
i_Bi=>i_b(2), i_CarryIn=>c(2),   o_Sum=>o_sum(2),
o_CarryOut=>c(3) );
    FA3: onebitadder port map ( i_Ai=>i_a(3),
i_Bi=>i_b(3), i_CarryIn=>c(3),   o_Sum=>o_sum(3),
o_CarryOut=>o_cout );
end architecture structural;
```

hexBcdDecoder.vhdl

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity hex_bcd_decoder is
    port (
        i_hex  : in  std_logic_vector(3 downto 0);
        o_bcd1 : out std_logic_vector(3 downto 0);  --
tens (left)
        o_bcd2 : out std_logic_vector(3 downto 0)   --
ones (right)
    );
end entity hex_bcd_decoder;

architecture structural of hex_bcd_decoder is
begin
    -- tens: 1 for 10 to 15
    o_bcd1(3 downto 1) <= "000";
    o_bcd1(0) <= (i_hex(3) and i_hex(2)) or (i_hex(3)
and i_hex(1));

    -- ones
    o_bcd2(3) <= i_hex(3) and not i_hex(2) and not
i_hex(1);
    o_bcd2(2) <= (not i_hex(3) and i_hex(2)) or
(i_hex(2) and i_hex(1));
    o_bcd2(1) <= (not i_hex(3) and i_hex(1)) or
(i_hex(3) and i_hex(2) and not i_hex(1));
    o_bcd2(0) <=
i_hex(0);
end architecture structural;
```

sevenSeg.vhdl

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity sevenSeg is
    port (
        i_clk      : in
std_logic;
        i_resetBar : in
std_logic;
        i_bcd1     : in  std_logic_vector(3 downto 0);
-- tens / left digit
        i_bcd2     : in  std_logic_vector(3 downto 0);
-- ones / right digit
        o_an       : out std_logic_vector(7 downto 0);
        o_seg      : out std_logic_vector(6 downto 0)
    );
end entity sevenSeg;

architecture structural of sevenSeg is

    component enardFF_2 is
        port (
            i_resetBar : in  std_logic;
            i_d        : in  std_logic;
            i_enable   : in  std_logic;
            i_clock    : in  std_logic;
            o_q        : out std_logic;
            o_qbar     : out std_logic
        );
    end component;

    component dec_7seg
is
        port (
            i_hexDigit  : in  std_logic_vector(3 downto
0);
            o_segment_a, o_segment_b, o_segment_c,
o_segment_d, o_segment_e,
            o_segment_f, o_segment_g : out std_logic
        );
    end component;

    signal sel     : std_logic;
    signal sel_n   :
std_logic;
    signal sel_bcd : std_logic_vector(3 downto 0);

begin

    -- toggles every clock: sel = 0, 1, 0, 1 ...
    FF: enardFF_2
        port map (
            i_resetBar => i_resetBar,
            i_d        =>
sel_n,
            i_enable   => '1',
            i_clock    => i_clk,
            o_q        => sel,
            o_qbar     =>
sel_n
        );

    -- pick the digit to show
    sel_bcd(0) <= i_bcd2(0) when sel = '0' else
i_bcd1(0);
    sel_bcd(1) <= i_bcd2(1) when sel = '0' else
i_bcd1(1);
    sel_bcd(2) <= i_bcd2(2) when sel = '0' else
i_bcd1(2);
    sel_bcd(3) <= i_bcd2(3) when sel = '0' else
i_bcd1(3);

    -- sel = 0: right digit (ones), sel = 1: left digit
(tens)
    o_an <= "11111110" when sel = '0' else "11111101";

    DEC: dec_7seg
        port map
(
            i_hexDigit  => sel_bcd,
            o_segment_a => o_seg(0),
            o_segment_b => o_seg(1),
            o_segment_c => o_seg(2),
            o_segment_d => o_seg(3),
            o_segment_e => o_seg(4),
            o_segment_f => o_seg(5),
            o_segment_g => o_seg(6)
        );

end architecture structural;
```